



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

SISTEME DISTRIBUITE

Assignment 1

Request-Reply Communication

Nume: Cojocaru Ana-Cătălina

Grupa: 30643

Profesori îndrumători:

Marcel Antal, Vasile Ofrim

2024-2025

Contents

1) Conceptual architecture of the distributed system.....	3
2) UML Deployment diagram	4
3) Fisierile docker	6
4) Readme file containing build and execution considerations	6

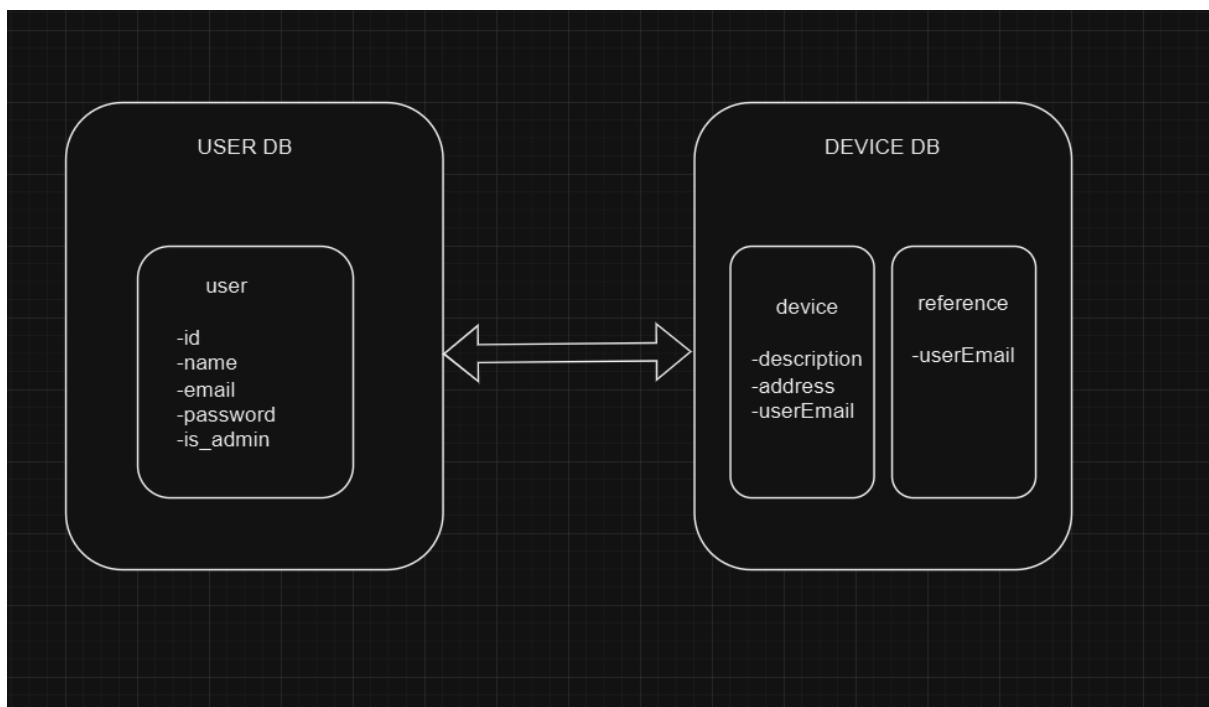
1) Conceptual architecture of the distributed system

Primul assignment constă într-o platformă online destinată gestionării clienților și a dispozitivelor. Sistemul poate fi accesat de două tipuri de utilizatori după un proces de autentificare: administrator și clienți.

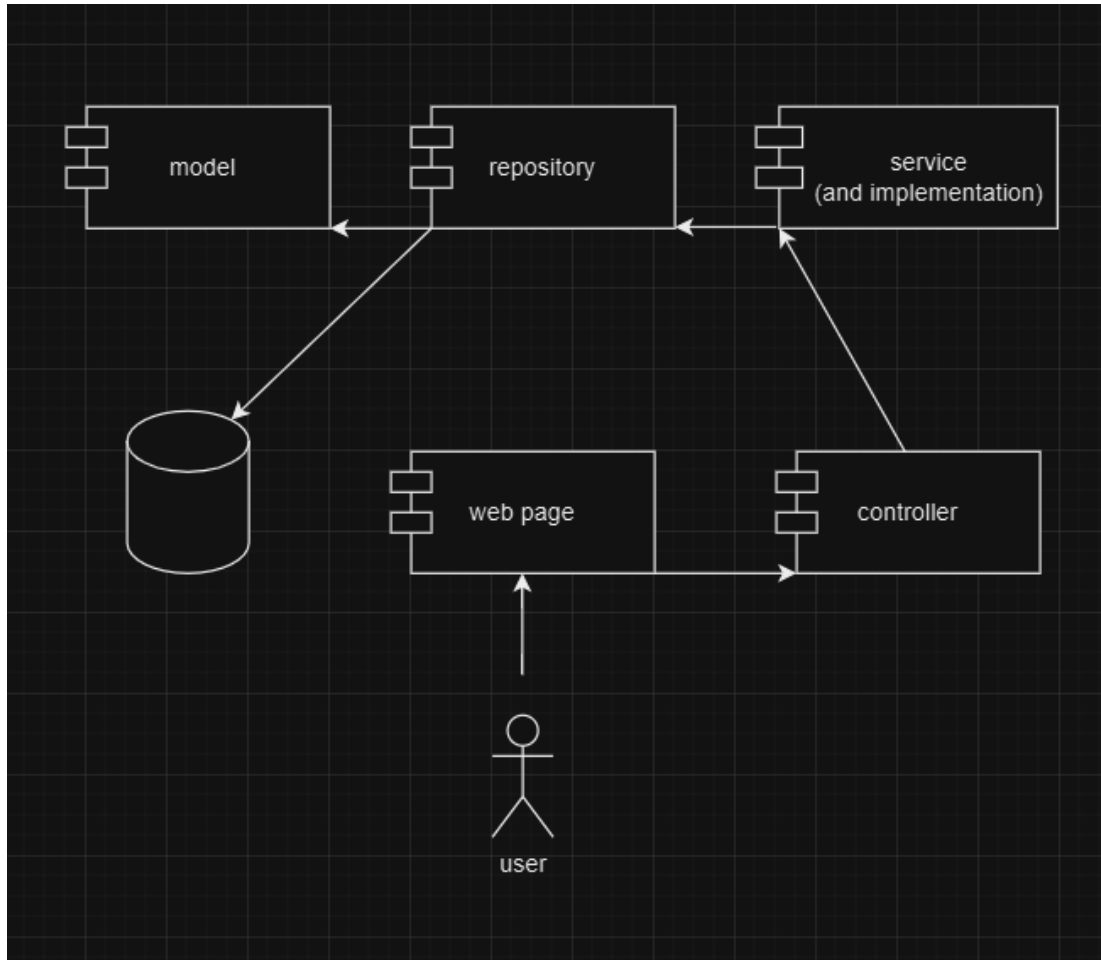
Administratorul are capacitatea de a efectua operațiuni CRUD (Creare, Citire, Actualizare, Ștergere) pe conturile clienților, care sunt definite prin ID (UUID generat care însă nu se poate schimba), nume, email și parola (care nu este afișată). De asemenea, administratorul poate gestiona dispozitivele înregistrate, care au un indice de consumului de energie (mhec). Aceste dispozitive sunt definite prin ID, descriere, adresă (locație fizică), consum maxim de energie și email-ul utilizatorilor asociați. Această legătură între User și Device se face printr-o tabelă adițională, User Reference ce reține email-ul userilor ce au Device-uri.

Administratorul poate realiza maparea clienților la dispozitive (fiecare client putând deține unul sau mai multe dispozitive).

Fiecare client are acces la o interfață prin care poate vizualiza dispozitivele deținute, consumul lor de energie actual și de asemenea poate să își adauge dispozitive și să le modifice pe cele existente.



2) UML Deployment diagram



Am ales o arhitectură server-client pentru acest proiect, împărțindu-l în două componente principale: Backend și Frontend.

Backend

Pentru partea de Backend, am folosit **Java** împreună cu framework-ul **Spring**. Spring este un framework foarte popular, apreciat pentru modul în care simplifică dezvoltarea aplicațiilor Java.

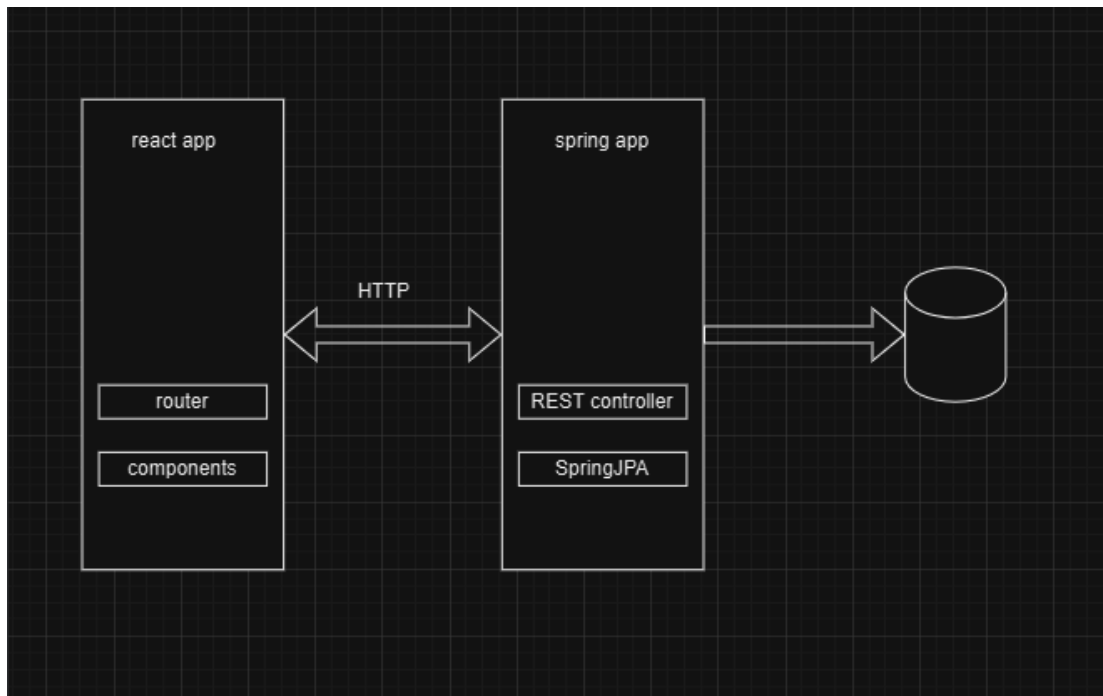
Am creat două aplicații ce comunică între ele prin REST.

Fiecare conține componentele și implementările necesare respectivelor baze de date, într-o arhitectură de tip layer.

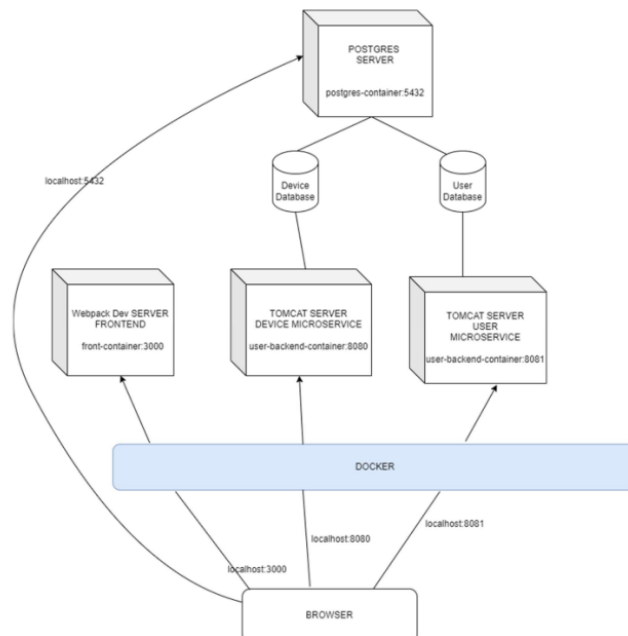
Frontend

Pentru partea de Frontend, am ales să folosesc **React**, o bibliotecă JavaScript care permite construirea de interfețe de utilizator dinamice și responsive. React permite dezvoltarea aplicațiilor sub formă de componente reutilizabile, ceea ce ajută la menținerea unui cod curat și modular.

Aceasta preia requesturile utilizatorilor și le transmite backend-urilor pentru procesare.

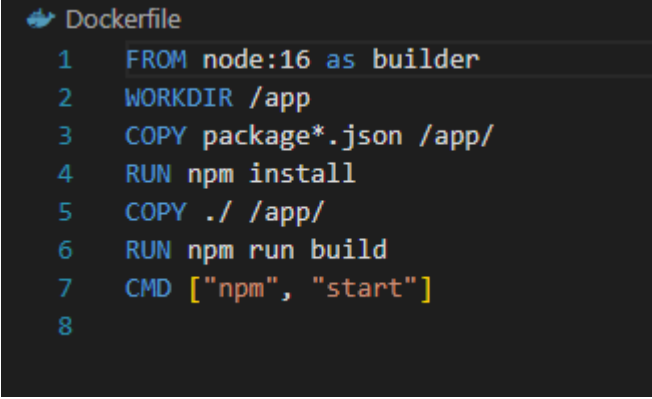


Diagramele arată comunicarea dintre partea de server și partea de client a aplicațiilor Web. Backend-ul comunica cu frontend-ul prin data transfer objects (dto). Serverul de Java este legat de baza de date cu care comunica prin intermediul repository-urilor și ia informații de la aceasta prin interogări. Clientul (browserul web) comunica cu serverul prin intermediul requesturilor de tip HTTP.



3) Fisierul docker

```
1 FROM maven:3.8.8-eclipse-temurin-17 AS builder
2
3 COPY ./src/ /root/src
4 COPY ./pom.xml /root/
5 COPY ./checkstyle.xml /root/
6 WORKDIR /root
7 RUN mvn clean package -DskipTests
8
9 FROM openjdk:17
10
11 ENV TZ=UTC
12
13 COPY --from=builder /root/target/ds-2020-0.0.1-SNAPSHOT.jar /app/app.jar
14
15 EXPOSE 8081:8081
16
17 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
18
```

A screenshot of a code editor showing a Dockerfile. The title bar says 'Dockerfile'. The content is as follows:

```
1 FROM node:16 as builder
2 WORKDIR /app
3 COPY package*.json /app/
4 RUN npm install
5 COPY ./ /app/
6 RUN npm run build
7 CMD ["npm", "start"]
8
```

4) Readme file containing build and execution considerations

Daca se doreste rularea aplicatiei local, se creeaza 2 directoare pentru backend (fiecare microserviciu va avea proiectul propriu) respectiv unul pentru frontend si se cloneaza cele 2 repository git aferente. Apoi se deschide proiectul de backend si se ruleaza, iar deschiderea aplicatiei de frontend se face cu comanda : npm start.

Necesare pentru rulare: IntelliJ, pgAdmin, node16, java17. VisualStudio (optional)